

Тема 13 Транзакции

Изменения БД обусловлены событиями во внешнем мире, например, перевод денег со счета на счет.

При этом одно внешнее событие приводит обычно к нескольким изменениям в БД.

Чтобы не нарушить целостность БД надо все изменения выполнить как единое целое.

Поэтому изменения в БД, вызванные одним событием, надо вносить по принципу: «либо все, либо ничего»



Транзакция является рабочей единицей работы с базой данных (далее – БД). Это последовательность операций, выполняемых в логическом порядке пользователем, либо программой, которая работает с БД.

Мы можем сказать, что транзакция – это распространение изменений в БД. Например, если мы создаём, изменяем или удаляем запись, то мы выполняем транзакцию. Крайне важно контролировать транзакции для гарантирования.

Транзакция представляет собой группу запросов SQL, обрабатываемых атомарно, то есть как единое целое. Если подсистема базы данных может выполнить всю группу запросов, она делает это, но если какой-либо запрос не может быть выполнен в результате сбоя или по иной причине, ни один запрос группы не будет выполнен.

Транзакция переводит БД из одного целостного состояния в другое.



Механизм транзакций обеспечивает:

1. Сохранение целостности данных;
2. Параллельную работу пользователей;
3. Восстановление данных при отказах и сбоях;

При отказах информационной системы желательно сохранить сведения о попытке выполнения транзакции. Если при этом транзакция была выполнена частично, то уже выполненную часть операций необходимо **откатить**. После восстановления системы транзакция должна быть выполнена повторно.

Основные концепции транзакции описываются аббревиатурой ACID – Atomicity, Consistency, Isolation, Durability (Атомарность, Согласованность, Изолированность, Долговечность).



Атомарность

Атомарность гарантирует, что любая транзакция будет зафиксирована только целиком (полностью). Если одна из операций в последовательности не будет выполнена, то вся транзакция будет отменена. Тут вводится понятие “отката” (rollback). Т.е. внутри последовательности будут происходить определённые изменения, но по итогу все они будут отменены (“откачены”) и по итогу пользователь не увидит никаких изменений.

Согласованность

Это означает, что любая завершённая транзакция (транзакция, которая достигла завершения транзакции – end of transaction) фиксирует только допустимые результаты. Например, при переводе денег с одного счёта на другой, в случае, если деньги ушли с одного счёта, они должны прийти на другой (это и есть согласованность системы). Списание и зачисление – это две разные транзакции, поэтому первая транзакция пройдёт без ошибок, а второй просто не будет. Именно поэтому крайне важно учитывать это свойство и поддерживать баланс системы.

Изолированность

Каждая транзакция должна быть изолирована от других, т.е. её результат не должен зависеть от выполнения других параллельных транзакций. На практике, изолированность крайне труднодостижимая вещь, поэтому здесь вводится понятие “уровни изолированности” (транзакция изолируется не полностью).

Долговечность

Эта концепция гарантирует, что если мы получили подтверждение о выполнении транзакции, то изменения, вызванные этой транзакцией не должны быть отменены из-за сбоя системы (например, отключение электропитания).

Управление транзакциями

Для управления транзакциями используются следующие команды:

- **COMMIT**

Сохраняет изменения

- **ROLLBACK**

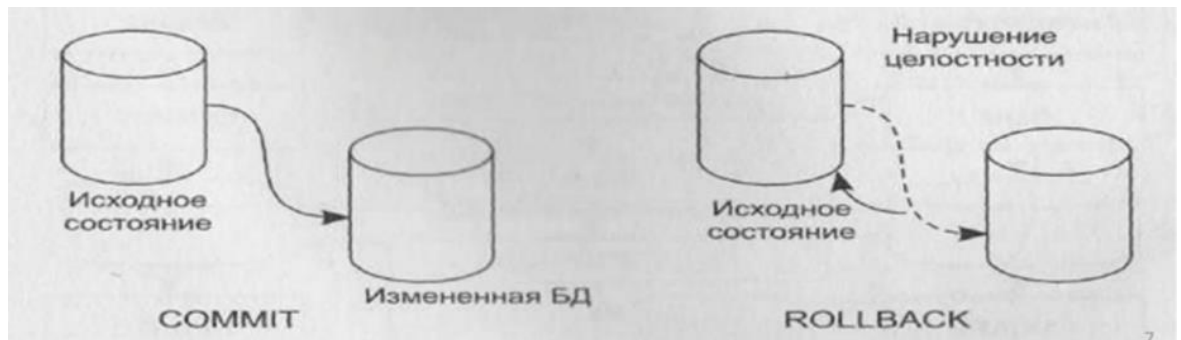
Откатывает (отменяет) изменения

- **SAVEPOINT**

Создаёт точку к которой группа транзакций может откатиться

- **SET TRANSACTION**

Размещает имя транзакции.



Команды управление транзакциями используются только для DML команд: INSERT, UPDATE, DELETE. Они не могут быть использованы во время создания, изменения или удаления таблицы.

Примеры:

Перед началом выполните следующую команду, для того, чтобы отключить автоматическое выполнение транзакции:

```
mysql> SET autocommit=0;
```

Предположим, что у нас есть таблица **developers**, которая содержит следующие записи:

ID	NAME	SPECIALTY	EXPERIENCE	SALARY
1	Eugene Suleimanov	Java	2	2500
2	Peter Romanenko	Java	3	3500
3	Andrei Komarov	C++	3	2500
4	Konstantin Geiko	C#	2	2000
5	Asya Suleimanova	UI/UX	2	1800
7	Ivan Ivanov	C#	1	900
8	Ludmila Geiko	UI/UX	2	1800

Удалим всех C++ разработчиков с помощью следующей команды:

```
mysql> DELETE FROM developers  
WHERE SPECIALTY = 'C++';  
mysql> COMMIT;
```

В результате выполнения данного запроса наша таблица будет содержать следующие записи:

ID	NAME	SPECIALTY	EXPERIENCE	SALARY
1	Eugene Suleimanov	Java	2	2500
2	Peter Romanenko	Java	3	3500
4	Konstantin Geiko	C#	2	2000
5	Asya Suleimanova	UI/UX	2	1800
7	Ivan Ivanov	C#	1	900
8	Ludmila Geiko	UI/UX	2	1800

Теперь выполнив команду **ROLLBACK**:

```
mysql> ROLLBACK;
```

После выполнения данной команды таблица содержит следующие данные:

ID	NAME	SPECIALTY	EXPERIENCE	SALARY
1	Eugene Suleimanov	Java	2	2500
2	Peter Romanenko	Java	3	3500
3	Andrei Komarov	C++	3	2500
4	Konstantin Geiko	C#	2	2000
5	Asya Suleimanova	UI/UX	2	1800
6	Ludmila Geiko	UI/UX	2	1800
7	Ivan Ivanov	C#	1	900

Как мы видим, запись C++ разработчика вновь в таблице.

Теперь разберемся с командой SAVEPOINT.

Для начала создадим точку сохранения, используя следующий запрос:

```
mysql> SAVEPOINT SP;
```

Теперь выполним следующие запросы:

```
mysql> DELETE FROM developers WHERE ID =7;
```

Query OK, 1 row affected (0.00 sec)

```
mysql> DELETE FROM developers WHERE ID = 6;
```

Query OK, 1 row affected (0.02 sec)

```
mysql> DELETE FROM developers WHERE ID = 5;
```

Query OK, 1 row affected (0.00 sec)

На данный момент наша таблица содержит следующие записи:

ID	NAME	SPECIALTY	EXPERIENCE	SALARY
1	Eugene Suleimanov	Java	2	2500
2	Peter Romanenko	Java	3	3500
3	Andrei Komarov	C++	3	2500
4	Konstantin Geiko	C#	2	2000

Теперь вернёмся к точке сохранения SP1 с помощью команды:

```
mysql> ROLLBACK TO SP1;
```

После выполнения данного запроса, наша таблица будет хранить следующие записи:

ID	NAME	SPECIALTY	EXPERIENCE	SALARY
1	Eugene Suleimanov	Java	2	2500
2	Peter Romanenko	Java	3	3500
3	Andrei Komarov	C++	3	2500
4	Konstantin Geiko	C#	2	2000
5	Asya Suleimanova	UI/UX	2	1800
6	Ludmila Geiko	UI/UX	2	1800
7	Ivan Ivanov	C#	1	900

Как видим, мы откатились к состоянию таблицы на момент создания точки сохранения SP1.

Теперь, когда нам больше не нужна данная точка сохранения, мы можем её освободить:

```
mysql> RELEASE SAVEPOINT SP1;
```

В завершение рассмотрим команду **SET TRANSACTION**, которая используется для того, чтобы инициировать транзакцию БД. Данная команда, позволяет нам определить характеристики транзакции.

Например, если мы хотим, указать, что транзакция предназначена только для чтения, то мы должны использовать следующий запрос:

```
SET TRANSACTION READ ONLY;
```

Если же мы хотим, чтобы транзакция позволяла выполнять запись данных, то запрос будет иметь вид, указанный ниже:

```
SET TRANSACTION READ WRITE;
```


Журнал транзакций – системная структура, хранящая информацию об изменениях БД.

Общая структура журнала – последовательный файл, в котором фиксируется каждое изменение БД, происходящее в ходе выполнения транзакции.

Цель журнализации: обеспечение возможности восстановления согласованного состояния БД после любого сбоя.

Восстанавливается последнее по времени согласованное состояние БД.

Как работает журнал:

- Если пользователь выполняет инструкцию COMMIT в журнале отмечается конец транзакции.

- Если пользователь выполняет инструкцию ROLLBACK, СУБД обращается к журналу и извлекает из него исходные копии строк, измененных во время транзакции.

- Используя эти копии, СУБД возвращает строки в прежнее состояние и таким образом отменяет изменения, внесенные в ЮД в ходе транзакции.

- Варианты восстановления БД:

- Индивидуальный откат транзакции (только для незавершившихся транзакций).

- Восстановление после внезапной потери содержимого оперативной памяти (мягкий сбой).

- Восстановление после поломки основного внешнего носителя БД (жесткий сбой).

Варианты ведения журнала транзакций:

1. Протокол с отложенными обновлениями

<T1, ID_RECORD, атрибут, новое значение...>

2. Протокол с немедленными обновлениями

<T1, ID_RECORD, атрибут, новое значение, старое значение...>

Модель транзакций ANSI/ISO (SQL3)

Инструкция	Описание
START TRANSACTION	Устанавливает свойства новой транзакции и запускает транзакцию.
SET TRANSACTION	Устанавливает свойства очередной транзакции. Не влияет на текущую выполняемую транзакцию.
SET CONSTRAINTS	Устанавливает режим ограничений в текущей транзакции. Он определяет применяется ли ограничение немедленно или откладывается до более позднего времени.
SAVEPOINT	Создает точку сохранения в пределах транзакции. Откат текущей транзакции м.б. выполнен не к началу, а к транзакции, а к точке сохранения.
RELEASE SAVEPOINT	Освобождает точку сохранения и все ресурсы, которые она могла занимать.
COMMIT	Завершает успешную транзакцию и сохраняет в БД все внесенные изменения.
ROLLBACK	Выполняет откат изменений либо к началу транзакции, либо к заданной точке сохранения.